

**Neuro-Adaptive Network Flow Orchestrator: A Deep Reinforcement Learning
Approach for Proactive Congestion Control in Software-Defined Campus
Networks
(Internal)**

Dhruvin Hitesh Bhudia:169646

Supervisor: Mr. Tiberius Tabulu

An Informatics and Computer Science Project Proposal Submitted to the School of
Computing and Engineering Sciences in partial fulfillment of the requirements for the
award of a Degree in Bachelor of Science in Informatics and Computer Science

School of Computing and Engineering Sciences

Strathmore University

Nairobi, Kenya

June 2026

Declaration and Approval

The student declares that this project proposal has not been submitted to Strathmore University or any other University for the award of a Degree in Bachelor of Science in Informatics and Computer Science or any other Degree. To the best of the student's knowledge and belief, the research proposal contains no material previously published or written by another person except where due reference is made in the research proposal itself.

Student Admission Number: 169646

Sign: _____

Date: _____

Supervisor's Signature:

Sign: _____

Date: _____

Acknowledgment

The researcher would like to extend heartfelt gratitude to the assigned supervisor, Mr. Tiberius Tabulu, for the unwavering support, technical guidance, and expert mentorship provided throughout the conceptualization and formulation of this research proposal.

Abstract

University campus networks carry changing demands from learning platforms, cloud services and multimedia applications. Routing based on configured shortest-path costs can leave some links congested while alternative paths retain capacity. Deep Reinforcement Learning offers adaptive path selection, but its usefulness depends on reliable measurements, controlled route changes and fair evaluation. This project proposes the Neuro-Adaptive Network Flow Orchestrator, a software-defined framework for investigating proactive congestion mitigation in an emulated campus network. The study will characterize congestion indicators, critically compare existing solutions, integrate a Proximal Policy Optimization agent with guarded orchestration, and evaluate network performance and routing stability. Mininet, Open vSwitch and Ryu will support the operator testbed, while a matched Linux/FRRouting environment will support comparison with nominal-cost OSPF. PyTorch will provide the learning model, supported by a web-based operator dashboard. Evaluation will use repeated traffic scenarios, separate training and test data, and measurements of goodput, round-trip time, packet delivery, utilization, congestion duration and route changes. The expected contribution is a reproducible account of when adaptive routing helps, what operational safeguards it requires, and whether interventions precede defined congestion events under the tested conditions.

Table of Contents

Declaration and Approval.....	ii
Acknowledgment.....	iii
Abstract.....	iv
Table of Contents.....	v
List of Figures.....	viii
List of Tables.....	ix
List of Abbreviations.....	x
List of Equations.....	xi
Chapter One: Introduction.....	1
1.1 Background.....	1
1.2 Problem Statement.....	3
1.3 General Objective.....	4
1.3.1 Specific Objectives.....	5
1.3.2 Research Questions.....	5
1.4 Justification.....	5
1.5 Scope and Limitations.....	6
1.5.1 Scope of the Project.....	6
1.5.2 Limitations in the Project.....	7
1.5.3 Delimitations in the Project.....	7
Chapter Two: Literature Review.....	8
2.1 Introduction.....	8
2.2 Congestion Indicators and Measurement Challenges.....	8
2.2.1 Traffic Volume and Link Utilization.....	8
2.2.2 Queue Occupancy and Buffering.....	9
2.2.3 Delay, Goodput and Packet Loss.....	9
2.2.4 Implications for Research Question 1.....	10
2.3 Related Applications, Solutions and ML Implementations.....	10
2.3.1 OSPF and Non-Learning Traffic Engineering.....	10
2.3.2 Commercial Network Management Systems.....	11
2.3.2.1 Cisco Catalyst Center, Formerly Cisco DNA Center.....	11
2.3.2.2 Juniper Mist AI and Marvis.....	12

2.3.2.3 Arista CloudVision.....	13
2.3.3 Research Implementations for Intelligent Routing.....	14
2.3.3.1 RouteNet: Learning Network Performance.....	14
2.3.3.2 DRL–GNN Routing Optimization.....	15
2.3.3.3 ENERO: Hybrid DRL and Local Search.....	15
2.3.3.4 Graph Transformer Star Routing.....	16
2.3.4 Comparative Synthesis for Research Question 2.....	17
2.4 Gaps in Existing Applications and Solutions.....	18
2.4.1 Integration and Stability Requirements: Research Question 3.....	18
2.4.2 Evaluation and Evidence Requirements: Research Question 4.....	19
2.5 Conceptual Framework.....	20
Chapter Three: Development Methodology.....	23
3.1 Introduction.....	23
3.2 Research Paradigm.....	23
3.2.1 Data Acquisition.....	23
3.2.2 Data Preprocessing.....	24
3.2.3 Model Training.....	24
3.2.4 Model Validation and Testing.....	25
3.3 Software Development Methodology.....	27
3.3.1 Requirements Elicitation.....	29
3.3.2 System Prototyping.....	30
3.3.3 Algorithmic Integration.....	30
3.3.4 Iterative Refinement.....	30
3.3.5 Interface Construction.....	30
3.3.6 System Implementation.....	31
3.3.7 Testing and Deployment.....	31
3.4 System Analysis and Design.....	31
3.4.1 Use Case Diagram.....	31
3.4.2 Class Diagram.....	32
3.4.3 Entity Relationship Diagram.....	33
3.4.4 Database Schema.....	34
3.4.5 System Architecture.....	35
3.4.6 Wireframes.....	36

3.5 System Development Tools and Techniques.....	37
3.5.1 Mininet Network Emulator.....	38
3.5.2 Ryu Software-Defined Networking Framework.....	38
3.5.3 PyTorch Deep Learning Library.....	38
3.5.4 Deep Reinforcement Learning.....	39
3.5.5 Conditional Stability Assessment and Operational Safeguards.....	39
3.5.6 API and Event-Driven Integration.....	40
3.6 System Deliverables.....	40
3.6.1 System Documentation.....	40
3.6.2 Authentication Module.....	40
3.6.3 Telemetry Module.....	41
3.6.4 NANFO Cognitive Routing Module.....	41
3.6.4.1 The Actor–Critic Model.....	41
3.6.4.2 The Operational Safeguard Layer.....	41
3.6.5 Interactive Administrator Dashboard Module.....	41
References.....	42
Appendix.....	45
Appendix A1: Time Schedule.....	45
Appendix A2: Turn-It-In Report.....	46

List of Figures

Figure 2.1. Cisco Catalyst Center functional overview.....	12
Figure 2.2. Marvis evidence-assisted troubleshooting workflow.....	13
Figure 2.3. CloudVision telemetry and change-assurance workflow.....	14
Figure 2.4. Conceptual Framework Diagram.....	22
Figure 3.1. Hybrid Agile Development Lifecycle.....	29
Figure 3.2. NANFO operator use cases.....	32
Figure 3.3. Representative implemented classes and ownership.....	33
Figure 3.4. Core inventory and telemetry entity relationships.....	34
Figure 3.5. Current application and experiment architecture.....	36
Figure 3.6. Operator dashboard wireframe.....	37
Figure A1. Original project time schedule.....	45

List of Tables

Table 2.1: Comparison of existing systems and research implementations.....	17
Table 3.1: Representative relational schema specifications.....	34
Table 3.2: Software tools and experimental environment.....	37

List of Abbreviations

AI – Artificial Intelligence
API – Application Programming Interface
CPU – Central Processing Unit
CSV – Comma-Separated Values
DQN – Deep Q-Network
DRL – Deep Reinforcement Learning
ECMP – Equal-Cost Multipath
ERD – Entity Relationship Diagram
FRR – FRRouting
GNN – Graph Neural Network
GTSR – Graph Transformer Star Routing
ICMP – Internet Control Message Protocol
ICT – Information and Communication Technology
IP – Internet Protocol
JSON – JavaScript Object Notation
JSONB – Binary JSON (PostgreSQL storage type)
LAN – Local Area Network
ML – Machine Learning
NANFO – Neuro-Adaptive Network Flow Orchestrator
OSPF – Open Shortest Path First
PDF – Portable Document Format
PPO – Proximal Policy Optimization
QoS – Quality of Service
REST – Representational State Transfer
RTT – Round-Trip Time
SDN – Software-Defined Networking
SQL – Structured Query Language
UML – Unified Modeling Language
UUID – Universally Unique Identifier
WAN – Wide Area Network

List of Equations

Equation 3.1: Feature scaling.....	24
Equation 3.2: Measured routing reward.....	25
Equation 3.3: PPO clipped policy objective.....	25
Equation 3.4: Conditional next-step queue envelope.....	39
Equation 3.5: Queue function and drift acceptance condition.....	39

Chapter One: Introduction

1.1 Background

Digital technologies have become central to the delivery of teaching, learning, research and administrative services in higher-education institutions. Universities increasingly depend on learning management systems, digital libraries, cloud-hosted applications, video-conferencing platforms, online assessment systems and real-time collaboration tools. The African Union's Digital Education Strategy and Implementation Plan for 2023–2028 identifies digital infrastructure, including networks and devices, as a fundamental requirement for expanding the use of digital technologies in teaching, learning, research, assessment and educational administration. The strategy also recognizes the importance of developing sustainable school and campus networks across Africa (African Union Commission, 2022).

A campus network connects facilities such as teaching departments, laboratories, libraries and data centres to shared services. Traffic demand may change as users join online classes, transfer research files or access multimedia resources. These activities motivate the normal-load, burst and sustained-load scenarios used in this study; they are experimental workload categories rather than measurements of Strathmore University's operational network. When the arrival rate at a bottleneck exceeds its service rate, packets accumulate in queues. Persistent queues increase delay, and exhausted buffers can cause packet loss. Large buffers may therefore conceal congestion by preserving delivery at the expense of responsiveness (Gettys & Nichols, 2012).

Traffic engineering addresses how traffic is distributed across available network resources. Link utilization indicates the share of link capacity in use, queue occupancy describes waiting traffic, and goodput records useful data delivered to the receiver. Delay and packet loss reveal effects experienced by applications. These measurements are related but are not interchangeable: a low loss rate does not establish low queueing delay, and high utilization alone does not establish service failure. Their joint interpretation provides a stronger basis for congestion analysis than any single indicator (Gettys & Nichols, 2012; Kreutz et al., 2015).

Open Shortest Path First (OSPF) is a link-state routing protocol that computes shortest paths from a topology database and configured link costs. It responds to topology changes and supports equal-cost multipath forwarding; it is therefore not an entirely static protocol (Moy, 1998). However, ordinary OSPF does not automatically convert instantaneous queue occupancy or traffic demand into congestion-aware costs. A link can become heavily loaded without failing, leaving the selected shortest path unchanged while another path has spare capacity. This distinction motivates comparison with a clearly specified nominal-cost OSPF baseline rather than a general claim that OSPF cannot adapt.

Software-Defined Networking (SDN) separates control logic from packet forwarding and exposes programmable interfaces for installing forwarding rules. A logically centralized controller can combine topology information with device statistics and coordinate network-wide decisions (Kreutz et al., 2015). SDN supplies this visibility and programmability, but an application must still determine which action to take. A threshold-based application is comparatively simple to inspect, although its response depends on the selected thresholds and observation interval. More adaptive policies must account for the cost of changing routes as well as the benefit of using less-loaded paths.

Deep Reinforcement Learning (DRL) provides one way to learn such policies from interaction with an environment. A routing agent receives observations, selects an action and obtains a reward reflecting the resulting network performance. Research has explored topology-aware routing with graph neural networks, hybrid learning and search, and multi-agent packet routing (Almasan, Suárez-Varela, et al., 2022; Almasan, Xiao, et al., 2022; Li et al., 2025). These studies support investigation of learned traffic engineering, but their different traffic models, action spaces and evaluation environments prevent direct transfer of their reported gains to a campus deployment.

Two issues are particularly important for NANFO. First, an adaptive policy can change paths frequently even when its average delivery performance improves. Training stability, route stability and operational safety must therefore be evaluated separately. Proximal Policy Optimization (PPO) controls the policy-update objective, while Lyapunov-based safe reinforcement learning introduces constraints under

specified modelling assumptions; neither establishes unconditional safety for an unrelated network implementation (Schulman et al., 2017; Chow et al., 2018). Second, reported improvement can depend on random seeds, implementation details and experimental conditions, making repeatable baselines and held-out evaluation essential (Henderson et al., 2018).

The proposed Neuro-Adaptive Network Flow Orchestrator will investigate these issues through measured network emulation and guarded operator workflows. The application combines telemetry, topology visualization, simulation, intent approval, audit history and performance reporting. A separate PyTorch PPO agent selects between two candidate paths in the current learning experiment. The operator environment uses Mininet, Open vSwitch and Ryu, while matched routing comparisons use a separate Linux/FRRouting environment. These implemented components provide a foundation for the study, although the complete safeguarded autonomous control loop remains subject to further validation (Bhudia, 2026a, 2026b).

In this study, proactive intervention means a verified traffic-engineering action occurring before the onset of a predefined congestion event. Section 3.2.4 specifies how that timing will be assessed. Improved goodput after congestion has developed will count as mitigation, not evidence of prediction. This distinction allows the study to examine the title’s proactive aim without assuming that it has already been achieved.

1.2 Problem Statement

Campus networks must distribute changing traffic demand across finite link and queue capacity. When flows share a bottleneck, persistent queueing can increase delay and eventually reduce delivery performance, even where an alternative path remains available (Gettys & Nichols, 2012). Conventional OSPF establishes reachability using configured costs but does not, by default, optimize paths from instantaneous queue and utilization measurements (Moy, 1998). The resulting problem is the need to redistribute traffic according to observed conditions without introducing unnecessary forwarding changes.

Existing solutions address parts of this problem. Commercial platforms provide network assurance and controlled change workflows, while research systems demonstrate learned routing and topology generalization (Cisco Systems, 2026; Arista Networks, 2026; Almasan, Xiao, et al., 2022; Li et al., 2025). However, the reviewed evidence does not establish the combined effectiveness of a compact PPO routing policy, explicit operational safeguards and pre-congestion intervention in the proposed emulated campus setting. Performance gains reported for optical transport networks, wide-area networks or packet-level simulations do not answer that specific experimental question. Similarly, theoretical safety results require assumptions that must be checked against the implemented network (Chow et al., 2018).

The research gap is therefore a lack of directly applicable, reproducible evidence showing whether the proposed integration can reduce congestion-related degradation while controlling route changes under the study’s campus-style workloads. Without this evidence, neither improved delivery nor timely and stable intervention can be assumed. NANFO will address the gap through telemetry characterization, critical comparison of existing systems, integration of a guarded DRL-based prototype, and matched evaluation against nominal-cost OSPF and non-learning baselines. The intended contribution is bounded experimental evidence and an inspectable implementation, rather than a claim of universally superior routing or validated production autonomy.

1.3 General Objective

To develop and evaluate a stability-aware DRL flow orchestrator for proactive congestion mitigation in an emulated software-defined campus network by December 2026.

1.3.1 Specific Objectives

- i. To characterize five congestion indicators under normal, burst and sustained-load traffic in the emulated campus network by September 2026.
- ii. To compare at least three commercial systems and three research implementations against the project's congestion-management requirements by September 2026.
- iii. To integrate a PPO routing agent, operational safeguards and an administrator interface into a testable NANFO prototype by November 2026.
- iv. To evaluate NANFO against nominal-cost OSPF and non-learning baselines using repeated experiments and defined performance and stability metrics by December 2026.

The five indicators are link utilization, queue occupancy, round-trip time, goodput and packet loss. Objective 1 will produce measured scenario profiles; Objective 2 will produce the comparison and gap analysis in Chapter Two; Objective 3 will produce an integration-tested prototype; and Objective 4 will produce a reproducible evaluation report using the protocol in Section 3.2.4. The milestone months fall within the project schedule in Appendix A1 and describe completion targets, including validation of work already implemented.

1.3.2 Research Questions

- i. How do the five congestion indicators vary under normal, burst and sustained-load traffic in the emulated campus network?
- ii. How do existing commercial systems and research implementations meet the project's congestion-management requirements, and what limitations remain?
- iii. How can a PPO routing agent, operational safeguards and an administrator interface be integrated into a testable NANFO prototype?
- iv. How does NANFO compare with nominal-cost OSPF and non-learning baselines in repeated experiments using the defined performance and stability metrics?

1.4 Justification

The development of NANFO is justified from technical, academic and operational perspectives. Technically, SDN offers centralized visibility and programmable forwarding through which congestion-aware routing can be investigated (Kreutz et al.,

2015). Integrating a DRL policy with explicit safeguards will allow the study to examine both the usefulness of adaptive routing and the consequences of its decisions. The administrator interface will make telemetry, proposed changes and execution evidence available for review.

From an academic perspective, the study contributes to the growing body of research on the application of Deep Reinforcement Learning within Software-Defined Networking environments. The findings may provide insights into the effectiveness of adaptive traffic engineering approaches for congestion mitigation in resource-constrained campus networks, thereby informing future research and practical deployments in higher education institutions.

Economically, the project will investigate better use of available path capacity through a software-based testbed rather than assume that congestion can only be addressed by purchasing additional hardware. Mininet supports network experimentation using the resources of a single host, making controlled prototyping feasible without a dedicated physical campus laboratory (Lantz et al., 2010). Reduced procurement costs or longer backup-power operation are potential topics for later deployment studies, not established outcomes of the present experiments. The application's energy function currently estimates consumption; it does not switch off network ports or demonstrate energy savings (Bhudia, 2026b).

1.5 Scope and Limitations

1.5.1 Scope of the Project

The project covers the design, implementation and experimental evaluation of NANFO within an isolated wired-network environment. The bounded campus topology contains five forwarding nodes and four hosts, with two candidate paths for the foreground flow. The study will use synthetic, reproducible workload schedules and measured counters, queues and probes. The operator testbed uses Mininet, Open vSwitch and Ryu; the matched learning benchmark uses Linux forwarding and FRRouting so that routing methods share the same forwarding environment and offered traffic.

The application scope includes authentication, workspace-scoped network management, telemetry, topology and digital-twin views, deterministic what-if

simulation, guarded intent execution, timed overrides, audit records, measured alerts and CSV/PDF reporting. These functions support experimentation and operator inspection. The research contribution remains congestion-aware routing and its measured performance and stability; the broader application modules do not create additional research objectives (Bhudia, 2026b).

1.5.2 Limitations in the Project

The experiments do not use an authenticated replica of Strathmore University’s production topology or verified institutional traffic logs. Synthetic scenarios cannot represent every campus application, user behaviour or hardware effect. Processes share the development host, so scheduling and resource contention may affect timing. The selected PPO checkpoint has been evaluated with one trained initialization and a bounded scenario mix; those results do not establish generalization to larger networks, other hardware or all traffic distributions (Bhudia, 2026a).

Further limitations concern actuation and safety. The matched Linux/FRRouting experiment and the Ryu/OpenFlow operator environment have different control interfaces. Transferring a qualified policy between them requires compatibility and calibration checks. The deterministic simulator provides model-based estimates, and a one-step safety assessment depends on valid observation, arrival and service assumptions. Integrated safeguarded autonomy and early intervention across the full proposed workload set remain evaluation requirements (Bhudia, 2026b).

1.5.3 Delimitations in the Project

The study is deliberately restricted to wired Ethernet/IP traffic engineering, a bounded set of routing actions and an isolated emulated network. Wireless radio optimization, cellular slicing, intrusion-detection research, custom switch hardware and production deployment are excluded. Authentication and authorization remain application requirements even though intrusion detection is outside the research scope. Learned autonomous shaping, policing and physical power control are also excluded from the current PPO action space; manually approved shaping and policing are distinct operator capabilities.

Chapter Two: Literature Review

2.1 Introduction

This chapter follows the research questions in Section 1.3.2. Section 2.2 examines congestion indicators and measurement challenges for Research Question 1. Section 2.3 compares existing systems and research implementations for Research Question 2. Sections 2.4.1 and 2.4.2 derive the integration requirements and evaluation gap for Research Questions 3 and 4 respectively. Section 2.5 brings these findings together in the conceptual framework, following the proposal guide's sequence of challenges, related solutions, gaps and framework.

The review draws on primary research publications, protocol specifications and product documentation. Research publications provide evidence about algorithms, experimental conditions and reported outcomes. Vendor documents establish advertised functionality and deployment requirements, but do not provide an independent performance comparison with NANFO. Each source is therefore considered in relation to the type of claim it can support. The comparison focuses on telemetry, routing decisions, operational control, reproducibility and suitability for the bounded campus experiment.

2.2 Congestion Indicators and Measurement Challenges

2.2.1 Traffic Volume and Link Utilization

Traffic volume is the amount of data sent over a stated interval, whereas utilization expresses a measured transmission rate relative to the configured link capacity. A byte counter becomes useful for traffic analysis only when its measurement interval and the link capacity are known. SDN supports collection of port and flow statistics, allowing an application to compare traffic across links rather than examine each device in isolation (Kreutz et al., 2015). For NANFO, utilization will help identify whether a selected path is heavily loaded while another has available capacity.

Utilization must be interpreted alongside the observation interval. A long averaging window may hide a short burst, while a short window may be sensitive to counter timing and transient demand. Moreover, a busy link is not necessarily failing to meet application requirements. The experimental profiles will therefore compare normal

demand, temporary bursts and sustained overload using the same counter definitions and sampling schedule. Maximum link utilization will summarize the most heavily loaded measured link, while the underlying time series will retain the timing of the load change.

2.2.2 Queue Occupancy and Buffering

Queue occupancy describes the traffic waiting for service at an interface. It responds to the difference between arrivals and departures, making it informative when demand begins to exceed a link's transmission capacity. Gettys and Nichols (2012) show why large, persistently occupied buffers can create substantial delay even before heavy packet loss becomes visible. This phenomenon, known as bufferbloat, challenges any definition of network health based only on successful delivery.

Queue measurement also has practical limits. OpenFlow port statistics do not automatically provide every queue-depth measurement required by an experiment. The location of the queue, its unit and its configured capacity must be recorded. NANFO's emulation combines controller statistics with Linux queue-discipline measurements and endpoint probes. Queue backlog in bytes will not be treated as packet occupancy unless the conversion is justified. This separation helps distinguish a missing queue observation from an empty queue and avoids learning from falsely healthy states.

2.2.3 Delay, Goodput and Packet Loss

Delay, goodput and loss describe different consequences of congestion. Goodput measures useful payload delivered to the receiving application, while a sender's configured rate describes intended demand. Packet-delivery ratio compares received and transmitted packets for a defined flow and interval. Round-trip time (RTT) includes the forward path, return path and endpoint processing; it cannot be reported as a direct measurement of one-way delay. Queueing can increase RTT before delivery deteriorates, so these metrics should be interpreted jointly (Gettys & Nichols, 2012).

The measurement boundary is especially important for a short experiment. Stopping a receiver at the same instant as its sender may count packets still in flight as lost. Similarly, a failed probe provides evidence of an outage but does not produce a

measured RTT value. NANFO’s retained benchmark uses verified queue drainage before final delivery accounting and records unavailable latency separately from a numerical reward penalty (Bhudia, 2026a). These choices make the evidence more useful than a report containing throughput averages alone.

2.2.4 Implications for Research Question 1

The literature supports using utilization and queue occupancy to describe load and backlog, with RTT, goodput and loss to describe delivery consequences. It does not establish a universal ranking of which indicator is most important for every network. Their usefulness depends on workload, capacity, sampling and the routing action available. Research Question 1 will consequently characterize their behaviour across the three defined traffic families rather than claim causal importance from correlation alone. The resulting profiles will inform the agent’s observation contract and the congestion-event definition used in evaluation.

2.3 Related Applications, Solutions and ML Implementations

2.3.1 OSPF and Non-Learning Traffic Engineering

OSPF provides a well-defined conventional reference because its forwarding decisions can be reconstructed from configured link costs and the link-state database. Moy (1998) specifies shortest-path calculation, topology-change handling and equal-cost routes. These properties make the baseline reproducible, but they also define its scope: configured shortest paths are not necessarily the least congested paths. A fair comparison must record the costs and whether they are recomputed from measured capacity or remain nominal.

A load-aware heuristic offers a second useful reference. Unlike nominal-cost OSPF, a heuristic can respond directly to utilization and queue pressure. A hysteresis margin can discourage changes caused by small fluctuations, although its usefulness depends on the selected margin and workload. Fixed-route policies provide a further check: if the experiment favours one path throughout, a learned policy may appear successful simply by always choosing that path. NANFO will therefore compare PPO with OSPF, a documented load-aware heuristic and both constant-path policies. This design tests the value of learning rather than merely the availability of an alternative route.

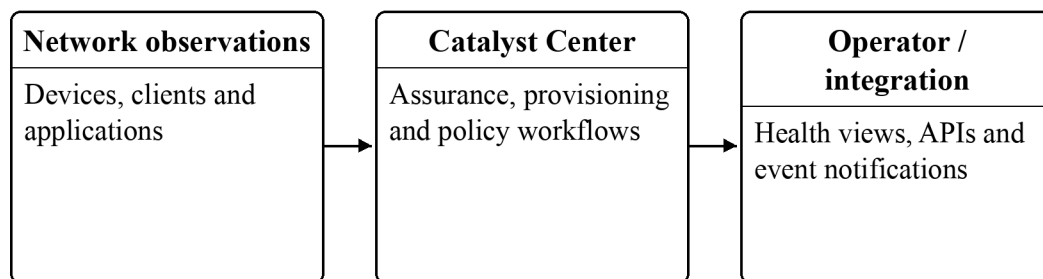
2.3.2 Commercial Network Management Systems

2.3.2.1 Cisco Catalyst Center, Formerly Cisco DNA Center

Cisco Catalyst Center combines device discovery, provisioning, policy automation and assurance for enterprise networks. Its assurance functions relate telemetry to device, client and application health, helping an administrator investigate service problems. The product also provides APIs and event notifications for integration with other operational tools. Cisco Systems (2026) documents visibility into third-party device reachability and topology, so the platform cannot accurately be described as having no multivendor support.

Its relevance to NANFO is the connection between network observations and an operator's troubleshooting workflow. A dashboard becomes more useful when a health indication can be traced to the affected device, client or path. However, third-party visibility should not be confused with identical provisioning and assurance support across all devices. Functionality depends on device compatibility and licensing. The data sheet describes subscription requirements but does not establish the acquisition cost for a particular Kenyan university; a claim of unaffordability would require a documented institutional comparison (Cisco Systems, 2026).

For this study, Catalyst Center is a functional comparator rather than an experimental routing baseline. Its documentation supports discussion of assurance and automation but does not expose the compact PPO policy, matched seed plan or route-change ablation needed for NANFO's research question. The design lesson is to present actionable telemetry and inspectable change records, while evaluating the routing algorithm separately.



Functional overview based on the cited product documentation

Figure 2.1. Cisco Catalyst Center functional overview

Note. Author’s schematic based on the assurance, automation and integration capabilities documented by Cisco Systems (2026).

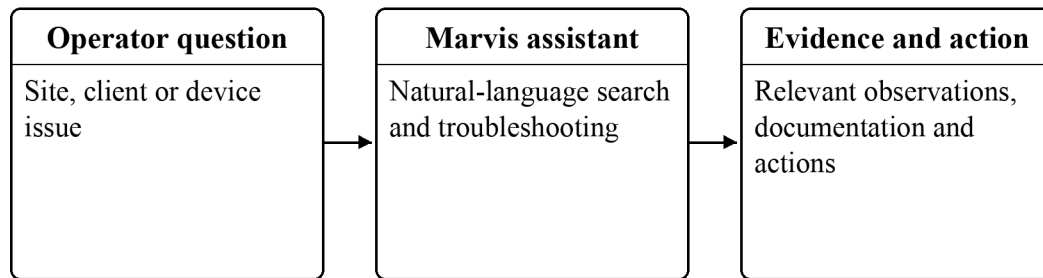
2.3.2.2 Juniper Mist AI and Marvis

Juniper’s Marvis Conversational Assistant provides natural-language access to network information and troubleshooting. Its documentation describes questions about sites, devices, clients and applications, together with links to relevant observations and pending actions. Access requires an appropriate subscription and user permissions (Juniper Networks, n.d.). These capabilities illustrate how an administrator can move from a service complaint to supporting network evidence without manually inspecting every device.

Marvis is relevant to NANFO’s explainability and human-in-the-loop requirements. Presenting a recommendation together with the affected resource and observation history can help an operator assess its relevance. However, the conversational-assistant documentation does not establish that natural-language troubleshooting performs the same task as DRL-based path selection. It also does not provide an independently reproducible campus-routing experiment against OSPF. Subscription requirements are a documented deployment consideration; assumptions about loss of local forwarding during a cloud outage would require separate evidence and are not made here.

NANFO will draw on the principle of evidence-linked operator interaction rather than attempt to reproduce a commercial conversational assistant. Its interface will expose measured telemetry, proposed actions and execution status through existing

application workflows. The comparison concerns how decisions are made understandable and reviewable, not a claim that NANFO offers broader enterprise assurance than Mist.



Functional overview based on the cited product documentation

Figure 2.2. Marvis evidence-assisted troubleshooting workflow

Note. Author’s schematic based on the conversational-assistant workflow documented by Juniper Networks (n.d.).

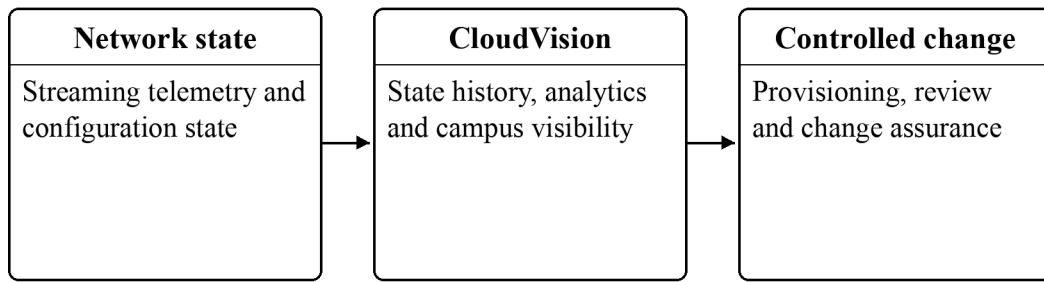
2.3.2.3 Arista CloudVision

Arista CloudVision combines streaming telemetry, network-wide state history, provisioning and change management. Its data sheet explicitly covers data-centre, campus, branch and wide-area environments, and describes on-premises and software-as-a-service deployment options (Arista Networks, 2026). It is therefore inaccurate to exclude CloudVision from campus networking or characterize it as necessarily dependent on a remote cloud service. Its state history and change-control functions are particularly relevant to understanding how an applied configuration affects network operation.

The platform offers a useful design reference for connecting an intended change to operational evidence. An administrator needs to distinguish a requested configuration, an applied configuration and the subsequently observed network condition. These distinctions inform NANFO’s approval records, configuration readback, measured path capture and rollback history. Nevertheless, the product documentation is not an independent evaluation of a publicly specified PPO policy under NANFO’s workloads. Subscription and device-feature compatibility must also be evaluated for

an actual deployment rather than reduced to an unsupported general cost claim (Arista Networks, 2026).

Across the three commercial platforms, the strongest common contribution is operational integration: telemetry is connected to troubleshooting, configuration or change assurance. The open question for NANFO is narrower. The study requires a reproducible path-selection experiment and evidence about the balance between performance improvement, timely intervention and route stability. Commercial capability descriptions establish relevant requirements but do not answer that experimental question.



Functional overview based on the cited product documentation

Figure 2.3. CloudVision telemetry and change-assurance workflow

Note. Author’s schematic based on the state-streaming and change-management capabilities documented by Arista Networks (2026).

2.3.3 Research Implementations for Intelligent Routing

2.3.3.1 RouteNet: Learning Network Performance

Rusek et al. (2019) introduce a graph neural network that learns relationships among topology, routing and input traffic to estimate end-to-end delay and jitter. The reported evaluation examines topologies, routing configurations and traffic not observed during training, with a worst reported coefficient of determination of 0.86. Its contribution is a learned network-performance model that can support optimization. It should not be described as a DRL routing agent that directly demonstrates throughput improvement over OSPF.

RouteNet is relevant because a controller needs to estimate the consequences of a candidate route, not simply display a topology. However, a predicted metric remains

model output. Its accuracy depends on the training distribution and the network properties represented in the input. The model does not by itself provide an authorization workflow or prove that an installed route is operationally safe. NANFO applies this distinction to its digital twin and simulator: predicted behaviour must remain distinguishable from measured traffic, and any use of prediction in decision-making requires validation against observations.

2.3.3.2 DRL–GNN Routing Optimization

Almasan, Suárez-Varela, et al. (2022) combine a graph neural network with a Deep Q-Network to route demands in an optical transport network. The agent evaluates a bounded set of candidate paths using link features and selects a path for each arriving demand. Its objective is to maximize the volume of traffic allocated over an episode. The evaluation includes 180 unseen synthetic topologies and 232 unseen real-world topologies, providing substantial evidence that graph-based representations can generalize beyond the training graph.

The experiment’s action and traffic assumptions are important. Demands are unsplittable, previously allocated demands cannot be rerouted, and accepted demands retain capacity until the episode ends. These conditions differ from NANFO’s active-flow rerouting under measured queues and changing demand. The reported allocation performance therefore supports the value of topology-aware representations but does not establish campus packet-delay reduction or the effect of repeated flow-rule changes. The design implication for NANFO is to keep the initial action space explicit and bounded, and to test any future topology generalization independently rather than infer it from this literature.

2.3.3.3 ENERO: Hybrid DRL and Local Search

ENERO uses a two-stage approach to wide-area traffic engineering: a graph-based DRL agent first proposes a routing configuration, and local search subsequently improves the solution. Almasan, Xiao, et al. (2022) report an average optimization time of 4.5 seconds for evaluated topologies with up to 100 links. The method concentrates effort on selected critical demands associated with heavily loaded links, illustrating how restricting the optimization problem can improve practical response time.

This is a closer traffic-engineering comparator than a generic assurance platform because it explicitly changes routing to reduce load concentration. It also challenges the assumption that a learning-only approach is always preferable: combining a learned starting point with a conventional optimizer can be effective. However, the reported optimization time is not equivalent to neural inference time or the complete time required to observe, authorize, install and verify a route in NANFO. WAN traffic matrices and segment-routing choices also differ from the bounded campus emulator. The relevant lesson is to compare against a competent heuristic and measure the complete control path alongside policy computation.

2.3.3.4 Graph Transformer Star Routing

Li et al. (2025) propose Graph Transformer Star Routing (GTSR), which combines multi-agent PPO with graph-transformer message passing and path-based readout. A virtual star node extends the agents' view beyond immediate neighbours. The implementation uses PyTorch and an OMNeT++ SDN simulation, with experiments involving Nsfnet and other Topology Zoo networks. Models trained on an original topology are tested after links are removed, with test observations excluded from weight updates.

The study evaluates end-to-end delay and packet loss against ECMP, other learned routing methods and architecture variants. Its setup specifies 1 Mbps links, 2 ms link delay, 512-byte packets, uniform traffic and a two-second control step, concentrating on heavy-load conditions. The results support improved robustness to the tested topology changes and show the value of ablation studies for explaining which model components help (Li et al., 2025). They are not a measured Mininet comparison against OSPF and should not be presented as one.

GTSR provides a recent, directly relevant example of PPO-based routing, but its multi-agent architecture is substantially different from NANFO's compact two-action policy. Uniform simulated traffic also differs from the proposed mixture of bursts and sustained demand. NANFO can borrow the separation of training and testing and the use of component ablations without claiming graph-transformer functionality. Its initial contribution will be a smaller, inspectable experiment linked to operator safeguards, with performance claims restricted to measured conditions.

2.3.4 Comparative Synthesis for Research Question 2

Table 2.1 compares the reviewed solutions using the same criteria: contribution, evidence and limitation relative to the proposed study. The limitation column identifies what each source does not establish for NANFO; it is not a claim that the product or algorithm has no value in its intended setting. The comparison shows that three commercial systems and four research implementations address different parts of the problem.

Table 2.1: Comparison of existing systems and research implementations

System and source	Contribution and evidence	Limitation and implication for NANFO
Catalyst Center (Cisco Systems, 2026)	Assurance, provisioning and third-party visibility documented by the vendor.	Feature and licensing dependencies; no matched open PPO benchmark. Adopt evidence-linked operator views.
Mist/Marvis (Juniper Networks, n.d.)	Natural-language troubleshooting linked to network observations.	Assistant functionality does not establish learned path-selection performance. Keep recommendations traceable.
CloudVision (Arista Networks, 2026)	State streaming, change control and campus support; cloud or on-premises options.	Product documentation is not an independent routing trial. Separate intended, applied and observed state.
RouteNet (Rusek et al., 2019)	GNN prediction of delay and jitter on unseen network configurations.	A performance model, not a DRL routing policy. Validate predictions against measurements.

System and source	Contribution and evidence	Limitation and implication for NANFO
DRL–GNN (Almasan, Suárez-Varela, et al., 2022)	DQN/GNN optical-demand allocation; extensive unseen-topology evaluation.	Accepted demands are not rerouted. Campus route churn requires separate measurement.
ENERO (Almasan, Xiao, et al., 2022)	DRL plus local search for WAN routing; reported 4.5-second mean optimization.	Different traffic/action model; optimizer time is not full actuation time. Include a strong heuristic baseline.
GTSR (Li et al., 2025)	Multi-agent PPO and graph transformers; simulated delay/loss and topology-change tests.	Uniform simulated workloads differ from measured campus bursts. Use held-out tests and component ablations.

Collectively, the literature supports adaptive traffic engineering while cautioning against a single undifferentiated claim of “intelligent networking”. Prediction, troubleshooting, path selection and verified actuation are distinct functions. None of the reviewed evaluations alone establishes the effectiveness of NANFO’s proposed integration in its particular emulator. This bounded finding motivates the design and evaluation gaps in the following section.

2.4 Gaps in Existing Applications and Solutions

2.4.1 Integration and Stability Requirements: Research Question 3

The first integration gap concerns the transition from a policy recommendation to a verified network action. Research routing agents specify state, action and reward, whereas the commercial platforms emphasize operational visibility and controlled change. NANFO needs both: a reproducible policy decision and an operator workflow that records the observation, proposed route, approval, installation result and subsequent readback. This requirement follows from the complementary contributions of the systems reviewed in Section 2.3, rather than from an assertion that no existing system supports safe change management.

The second gap is the distinction between stable learning and stable forwarding. PPO’s clipped objective discourages certain large policy updates but does not impose a hard limit on how often a running policy changes paths (Schulman et al., 2017). Chow et al. (2018) formulate safe reinforcement learning through constrained Markov decision processes and Lyapunov-derived constraints. Their guarantees depend on the specified problem and assumptions. Navarro-Alarcon et al. (2020) similarly establish an adaptive sensorimotor modelling method in a robotics context. That study supports the principle of explicit stability analysis, but its equations cannot be transferred to discrete network routing as an automatic proof of safety.

For NANFO, stability will therefore have an operational definition: observed route-change frequency, path-holding time and flow-rule update rate, assessed alongside delivery performance. The prototype will combine a route-change reward penalty with separate observation-age, minimum-hold and change-rate checks. These safeguards address different concerns: the reward discourages unnecessary changes during learning, while execution checks can reject an otherwise preferred action. Their contribution must be measured with an ablation, and rejected actions must remain visible in the audit trail. A configured one-step certificate will only support the assumptions and horizon it actually checks.

These findings answer the design aspect of Research Question 3 by defining an integration approach: measured telemetry feeds a versioned PPO policy; its proposal passes through explicit safeguards and authorization; the controller applies only an accepted action; and readback establishes what actually occurred. A digital-twin view and deterministic simulation can assist inspection, but neither substitutes for physical observation in the emulator. Full closed-loop validation remains necessary before the integrated prototype can claim safeguarded autonomous operation.

2.4.2 Evaluation and Evidence Requirements: Research Question 4

The evaluation gap is not simply a shortage of favourable performance results. Existing studies measure different outcomes under different conditions: RouteNet predicts delay and jitter; DRL–GNN allocates optical demands; ENERO optimizes WAN routing; and GTSR measures packet-routing behaviour in simulation (Rusek et al., 2019; Almasan, Suárez-Varela, et al., 2022; Almasan, Xiao, et al., 2022; Li et al.,

2025). Their percentages cannot be pooled into an expected improvement for NANFO. A matched experiment must establish its own performance effect and uncertainty.

Henderson et al. (2018) show that random seeds, implementation choices and reporting procedures materially affect DRL comparisons. For NANFO, the topology, offered traffic, queue settings, measurement duration and background-flow placement must be held constant across routing methods. Model selection must occur on validation data before reserved tests are opened. Repeated windows within one episode are not independent training runs, so the unit of comparison and the number of trained initializations must be reported explicitly.

Proactivity creates a further evidence requirement. High goodput or low average RTT after rerouting does not show that the action preceded congestion. The evaluation must retain timestamps for warning observations, applied route changes and congestion-event onset. A matched no-intervention run is needed to investigate whether a congestion event was avoided, since an event that never occurs has no observed onset time in the intervention run. Section 3.2.4 separates early action, successful avoidance, late mitigation and unsuccessful intervention.

Research Question 4 will therefore be addressed through paired performance comparisons, stability measurements, a safeguard ablation and explicit intervention timing. The study will report unfavourable and inconclusive outcomes as well as improvements. This protocol tests the combined claim of useful and controlled adaptation, while leaving broader claims about institutional cost savings, large-topology generalization and production safety for evidence from later studies.

2.5 Conceptual Framework

The conceptual framework for this study is founded upon the integration of Software-Defined Networking and Deep Reinforcement Learning to facilitate intelligent traffic engineering within campus network environments.

The framework begins with the collection of real-time network telemetry data from the data plane. These telemetry parameters include link utilization, queue occupancy, throughput measurements, packet loss statistics, and latency indicators. The collected

information is transmitted to a centralized SDN controller, which maintains a global view of the network topology and current operational state.

The controller forwards the processed telemetry information to the Neuro-Adaptive Network Flow Orchestrator (NANFO) cognitive engine. Within this component, the Deep Reinforcement Learning agent continuously evaluates network conditions and generates routing decisions aimed at minimizing congestion, reducing latency, improving throughput, and optimizing resource utilization.

The generated routing policies are subsequently validated through a neuro-adaptive stability mechanism before being communicated back to the SDN controller. The controller then installs the optimized flow rules within the network switches, resulting in dynamic traffic redistribution across available paths.

The expected outcomes of the framework include improved network performance, enhanced congestion mitigation, reduced latency, improved bandwidth utilization, and more efficient use of existing networking infrastructure. The relationship among these variables is illustrated in Figure 2.4.

For the current implementation, Figure 2.4 represents the intended closed loop. Its “Network” actor is the emulated data plane, and its “DRL engine” is the separately executed PPO policy. The trainable feedback path applies during controlled training; evaluation uses frozen weights. “Neuro-Adaptive Validation” refers to the explicit operational checks and conditional safety assessment described in Section 3.5.5, not an unconditional Lyapunov guarantee. Installation is conditional on acceptance and authorization; a rejected proposal retains the permitted configuration. The baseline mode represents a separately configured comparison run, rather than an assertion that the application can freely switch between incompatible forwarding environments.

The independent experimental factors are the routing method, traffic scenario and safeguard configuration. The dependent outcomes are the defined performance, intervention-timing and routing-stability measures. Topology, capacities, workload seeds and measurement rules are controlled factors. This interpretation connects the preserved framework to all four research questions without treating its expected outputs as completed results.

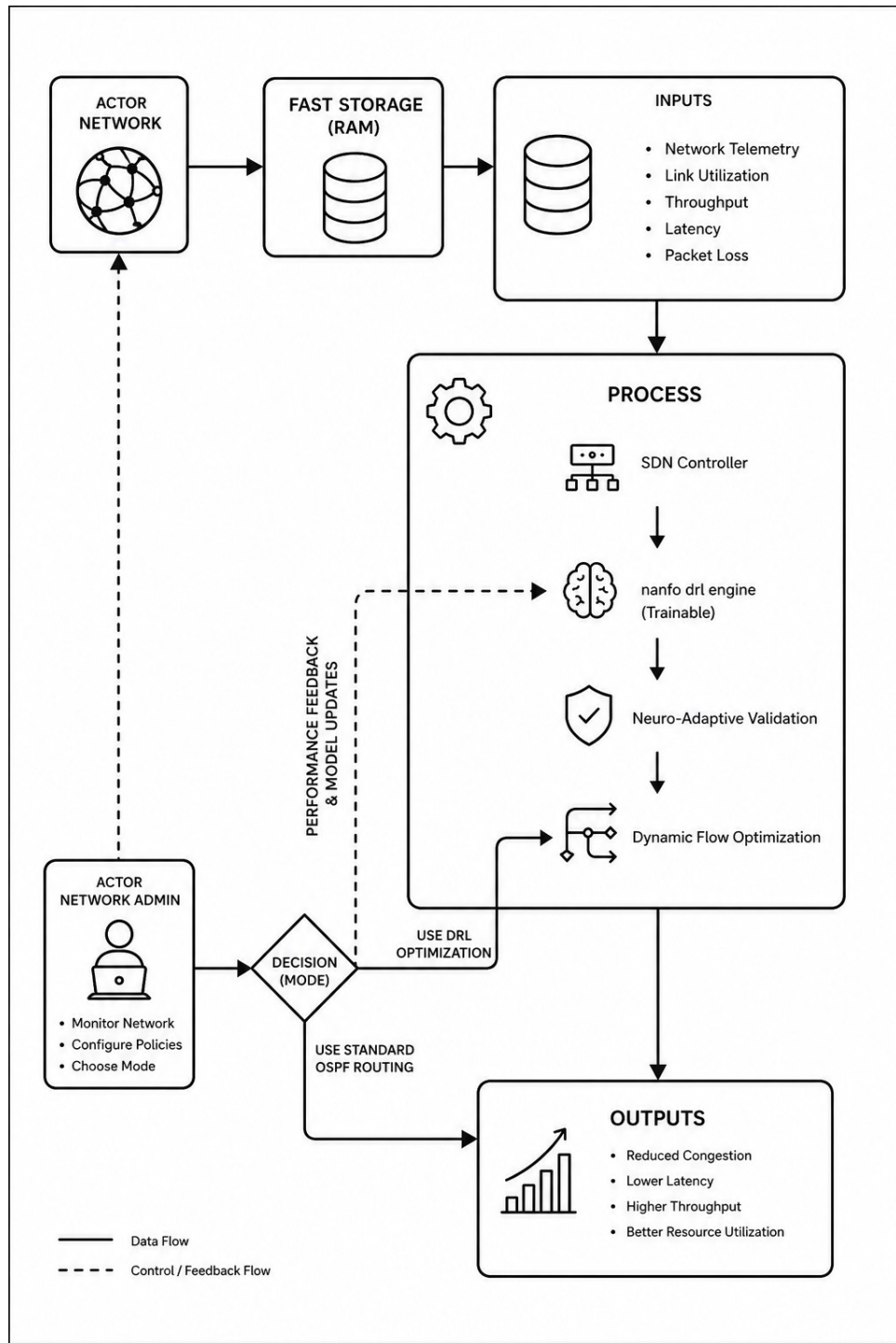


Figure 2.4. Conceptual Framework Diagram

Note. Conceptual framework retained from the assessed proposal; implementation interpretation is provided above.

Chapter Three: Development Methodology

3.1 Introduction

This chapter describes how the study will acquire network observations, prepare model inputs, train the routing policy and evaluate the prototype. It retains the experimental research approach and Hybrid Agile development method, while specifying the application components and measurement procedures used in the current implementation. The methodology links each research objective to an observable output and distinguishes completed prototype evidence from the remaining evaluation work.

3.2 Research Paradigm

The study adopts an experimental, quantitative approach. Routing method and safeguard configuration will be varied within a controlled emulated environment, while network outcomes are measured using a common protocol. This approach is appropriate because the central question concerns the effect of a software artefact on observable network behaviour. Separate training, validation and test stages will reduce the risk of reporting improvements caused by favourable seed selection or repeated tuning on test results (Henderson et al., 2018).

3.2.1 Data Acquisition

Data will be generated within the bounded five-node, four-host campus-style topology. The operator environment uses Mininet, Open vSwitch and Ryu, and the comparative learning environment uses matched Linux forwarding with FRRouting for OSPF. Both environments have explicit topology and workload definitions, but their control interfaces are not interchangeable. Institutional topology diagrams and historical flow logs are not assumed to have been supplied; synthetic scenarios will be labelled accordingly (Bhudia, 2026b).

Acquisition will combine port counters, Linux queue-discipline observations, endpoint transmission and reception records, and ICMP probes. Each observation will retain its timestamp, interval, unit, source, network identity and experiment identity. The normal, burst and sustained-load families will vary offered demand against documented capacity. Capacity-impairment experiments will also reproduce the

retained benchmark with either candidate path constrained. Methods will use matching workload seeds and background-traffic placement, with explicit reset and queue-drain procedures between comparable runs.

3.2.2 Data Preprocessing

Preprocessing will validate completeness, finite values, measurement intervals and correspondence between the observation and the active experiment. Missing data will not be silently interpreted as zero load. The current model uses thirteen numerical features: utilization and queue occupancy for each candidate path; RTT, loss and goodput; requested foreground and background rates; elapsed time since the previous route change; actual foreground rate; and the two path capacities. An RTT-availability flag and two previous-action indicators produce a sixteen-value input vector (Bhudia, 2026b).

The implemented numerical transform is logarithmic scaling with fixed positive feature scales. Equation 3.1 defines the transformation, where x_j is a non-negative measured feature and s_j is its documented scale. For an unavailable RTT, the numerical placeholder is zero and the availability flag is false. Other incomplete required measurements invalidate the learning transition. The topology is fixed by the experiment contract rather than concatenated into the current model input.

$$z_j = \ln(1 + x_j / s_j), \text{ where } s_j > 0$$

Equation 3.1: Feature scaling

Feature scales, action mappings and reward coefficients will be retained with the model checkpoint. They will not be fitted using reserved test observations. The current stationary benchmark uses one observed frame; a claim of forecasting future traffic would require an additional, explicitly evaluated temporal model. Scenario labels, workload seeds and future phase information will not be provided as policy inputs.

3.2.3 Model Training

The routing task will be treated as a discrete-time sequential decision problem. The implemented PyTorch actor–critic model has separate thirty-two-unit hidden layers and a categorical actor with two candidate-route actions. The critic estimates expected return. PPO was selected because it supports this categorical policy and provides an

inspectable clipped policy-update objective; the implementation does not require a continuous link-weight or flow-splitting action space (Schulman et al., 2017).

The retained reward combines useful delivery with delay, loss, utilization, queue and route-change penalties. In Equation 3.2, G_t is measured goodput, F_t is actual offered foreground rate, D_t is RTT in milliseconds, L_t is loss fraction, U_t is maximum candidate-path utilization, Q_t is maximum candidate-path queue occupancy in packets, and C_t is one when the verified route changes. F_t must be positive. The displayed coefficients describe the current implementation and will remain frozen for evaluation of that checkpoint (Bhudia, 2026b).

$$r_t = \min(G_t / F_t, 1) - 0.2(D_t / 50) - L_t - 0.1U_t - 0.1(Q_t / 100) - 0.05C_t$$

Equation 3.2: Measured routing reward

A verified zero-reply outage retains a missing observed RTT. The training contract may apply its separately recorded 1,000 ms censored-delay penalty for that condition; this is a reward convention, not a fabricated latency measurement. Invalid or incomplete windows will not enter the optimizer. The policy-update objective follows Equation 3.3, where ρ_t is the new-to-old action-probability ratio, $\hat{A}_t$ is the advantage estimate and ϵ is the clipping parameter (Schulman et al., 2017).

$$L_{\text{clip}}(\theta) = \text{mean}_t[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \hat{A}_t)]$$

Equation 3.3: PPO clipped policy objective

Training will use fresh on-policy transitions, recorded optimizer updates and bounded experiment budgets. Candidate models will be selected using validation data before final tests are opened. Checkpoints will retain weights, configuration, feature contract, random-state information and provenance. The retained selected campaign completed 384 measured transitions and 24 updates; later refinement did not establish an accepted replacement. These records are existing development evidence, not a claim that the remaining evaluation has been completed (Bhudia, 2026a, 2026b).

3.2.4 Model Validation and Testing

The evaluation will compare frozen PPO, a load-aware heuristic, constant route 0, constant route 1 and nominal-cost OSPF. Each method will use the same graph, addressing, capacity schedule, queues, foreground demand and background placement

in the matched environment. OSPF costs will be documented and retained unchanged during the primary comparison. Any later capacity-aware OSPF comparison will be reported as a separate baseline rather than silently changing the original one.

The planned extension will use at least twelve fresh held-out workload seeds per traffic family, with training and validation seeds kept separate. Three independently trained model initializations are targeted to examine sensitivity to training randomness. The achieved number will be reported if the computing budget constrains this target. Method order will be recorded and balanced or shuffled in advance. Within-run measurements will first be aggregated by workload seed; they will not be counted as independent training runs. Paired differences, 95% confidence intervals, medians and tail RTT will be reported as appropriate, alongside the actual sample counts (Henderson et al., 2018).

The primary performance outcomes are received-payload goodput in Mbps and ICMP RTT in milliseconds. Secondary outcomes are packet-delivery ratio, loss fraction, maximum measured link utilization, congestion-event frequency and congestion duration. Stability outcomes are verified route changes per minute, mean path-holding time and flow-rule updates per minute. Model inference time, total observation-to-verification time and controller CPU/memory use will be reported separately. A gain in goodput will not be described as faster OSPF route computation. Metrics unavailable in a forwarding mode will be labelled unavailable rather than estimated from unrelated counters.

For the planned congestion analysis, an event will require at least three valid observations spanning ten seconds in which a documented limit remains breached: utilization at or above 85%, RTT at or above 100 ms, loss at or above 2%, or queue occupancy at or above 80% of its known packet capacity. The selected limit, observation source and unit will be fixed before a new evaluation campaign. Event onset will be assigned retrospectively to the first sample of the qualifying sustained breach; detection time will be recorded separately. Recovery will use the corresponding lower thresholds of 70% utilization, 70 ms RTT, 1% loss or 40% queue occupancy over the same sustained interval. These operational choices are aligned with the application’s measured-alert configuration; they are not universal congestion constants (Bhudia, 2026b).

Proactive timing will be assessed against paired no-intervention workload runs. Lead time will be the reference congestion-onset time minus the verified action time on the aligned workload timeline. A positive value indicates early intervention relative to that reference event. Successful avoidance additionally requires the intervention run to remain below the defined event condition during the corresponding risk interval. A route change after onset will be classified as mitigation. Invalid measurements and a reference run with no congestion event will be reported without inventing a lead time. This protocol assesses whether the current observation-driven policy acts early; it does not assume a separate traffic predictor exists.

The safeguard ablation will compare the same frozen policy with and without the additional route-hold and change-rate restrictions inside the isolated experiment. Base authorization, emergency-stop and resource-isolation controls will remain active. Comparisons will examine delivery, blocked proposals, actual route changes and recovery behaviour. Integration tests will separately cover stale observations, expired approvals, duplicate execution, cancellation, worker restart, rollback and emergency stop. Compatibility and calibrated-bound requirements must be met before a safeguarded autonomous intervention can count as completed evidence.

Existing evidence is narrower than this full protocol. The retained twelve-seed stationary capacity-impairment benchmark reported mean PPO goodput of 5.922 Mbps and ICMP RTT of 25.155 ms, compared with 3.946 Mbps and 104.090 ms for unchanged nominal-cost OSPF. It passed its predeclared paired goodput and RTT criterion. The result concerns one trained initialization, one method-session order and a specific 2/20 Mbps impairment mix. It does not establish proactive lead time, general superiority over capacity-aware OSPF, or integrated safeguarded autonomy (Bhudia, 2026a).

3.3 Software Development Methodology

The project retains a Hybrid Agile approach, combining time-boxed planning and review with a visual task board. The approach is adapted to an individual research project rather than presented as a full Scrum team implementation. It supports incremental work on telemetry, model behaviour, safeguards and the operator interface. Short review cycles are appropriate because measured experiments may

expose problems that require changes to the next development increment (Schwaber & Sutherland, 2020).

The Hybrid Agile methodology was selected because the development of machine learning-enabled networking systems involves continuous experimentation, iterative model refinement, and frequent performance evaluation. Unlike traditional linear development approaches, Hybrid Agile supports incremental implementation, rapid testing, and continuous improvement of both software components and learning algorithms, making it suitable for research-oriented system development.

Figure 3.1 shows five stages and their feedback relationships. First, requirements and the experimental contract establish the scope, baselines and acceptance criteria. Second, each implementation cycle produces a testable increment, such as measured telemetry or guarded execution. Third, integration tests and validation experiments examine whether the increment meets its criteria. Failed checks return to the development backlog with their evidence retained. Fourth, a frozen release undergoes reserved evaluation. Fifth, documentation brings together the evidence, operator guidance and remaining limitations. Once a final test has been opened, its results may be reported but cannot be used to tune that same evaluated model; further development requires a newly declared campaign.

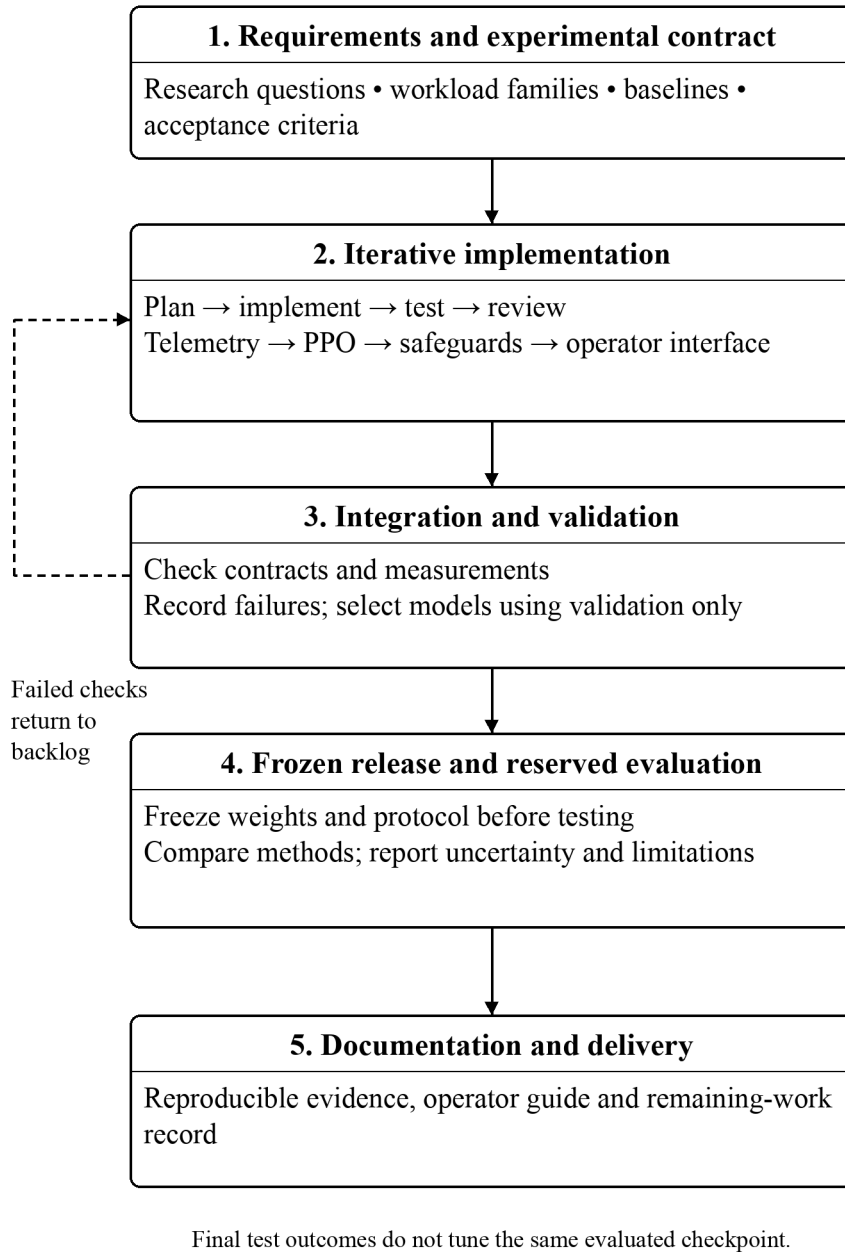


Figure 3.1. Hybrid Agile Development Lifecycle

Note. Author’s project-specific lifecycle, informed by the iterative planning and review principles of Schwaber and Sutherland (2020).

3.3.1 Requirements Elicitation

Requirements will be derived from the research questions, reviewed systems, application contracts and supervisor feedback. Functional requirements include measured telemetry, reproducible model evaluation, reviewable route proposals, authorized execution and verifiable recovery. Non-functional requirements include

scoped access, bounded resource use, traceable observations and repeatable deployment. Timing claims will be measured end to end rather than specified as unverified millisecond guarantees.

3.3.2 System Prototyping

The prototype will establish the bounded topology, controller connectivity, traffic generators and observation pipeline before linking model output to a control action. Baseline reachability and raw measurement checks will precede learning experiments. Separate runtime environments will keep the controller and PyTorch dependencies independently reproducible.

3.3.3 Algorithmic Integration

Integration will connect the PPO observation and action contracts to the application's governed decision workflow. The current learning client is a separate process rather than a model embedded inside Ryu. The implementation will retain a distinction between checkpoint qualification, diagnostic inference and permission to actuate. The relevant equations are the implemented reward and PPO objective, together with the separate conditional safety checks in Section 3.5.5.

3.3.4 Iterative Refinement

Refinement will use training and validation evidence to examine reward behaviour, useful directional decisions and stability trade-offs. A visual backlog will track defects, blocked acceptance criteria and completed increments. Changes to model architecture, workload assumptions or feature definitions will create a new recorded experiment version. Failed candidates will remain documented, and an incumbent checkpoint will only be replaced when the declared selection criteria are met.

3.3.5 Interface Construction

The React and TypeScript interface will present topology, telemetry, digital-twin views, simulations, intents, alerts, diagnostics and reports. It will distinguish a proposed route from a measured path and display observation age and execution status. Operator controls will follow current permissions, and requests to change training settings will not be labelled as changes to the frozen model's effective configuration.

3.3.6 System Implementation

The application uses a Python FastAPI modular monolith, with module-owned services and repositories and Redis Streams for event-driven integration. The frontend consumes authenticated application APIs and WebSocket updates. PostgreSQL stores relational records, Neo4j stores topology relationships, and Redis supports event delivery and short-lived coordination. Git history, documented contracts and review checks will track implementation changes (Bhudia, 2026b).

3.3.7 Testing and Deployment

The final phase will combine white-box unit testing, integration testing and black-box operator workflows. Tests will examine numerical calculations, authorization, event handling, measured execution and interface behaviour. Deployment will use the project's container configuration, health checks and operational documentation. A successful application test suite will be reported separately from a successful routing experiment or a live backup-and-restore acceptance test.

3.4 System Analysis and Design

The design artefacts describe the current implementation at a level appropriate to the proposal. Figures 3.2–3.6 connect operator use cases, representative classes, data ownership, deployment boundaries and interface structure. They are selective views of the prototype, while the versioned application contracts and migrations provide the detailed implementation record (Bhudia, 2026b).

3.4.1 Use Case Diagram

Figure 3.2 shows the administrator reviewing telemetry, evaluating a proposed change, approving execution, requesting restoration and generating reports. The emulated network supplies measurements and receives authorized controller actions. Model training and checkpoint evaluation are separate research operations. These relationships prevent a monitoring request or a diagnostic inference call from being interpreted as authority to alter forwarding.

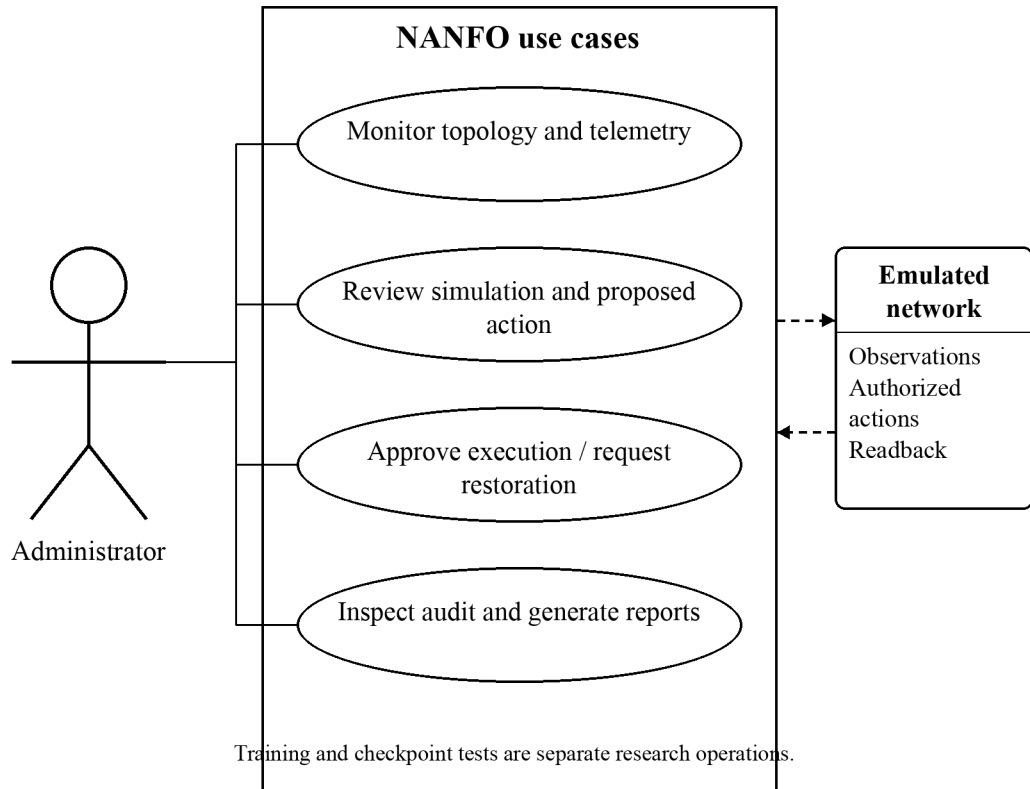
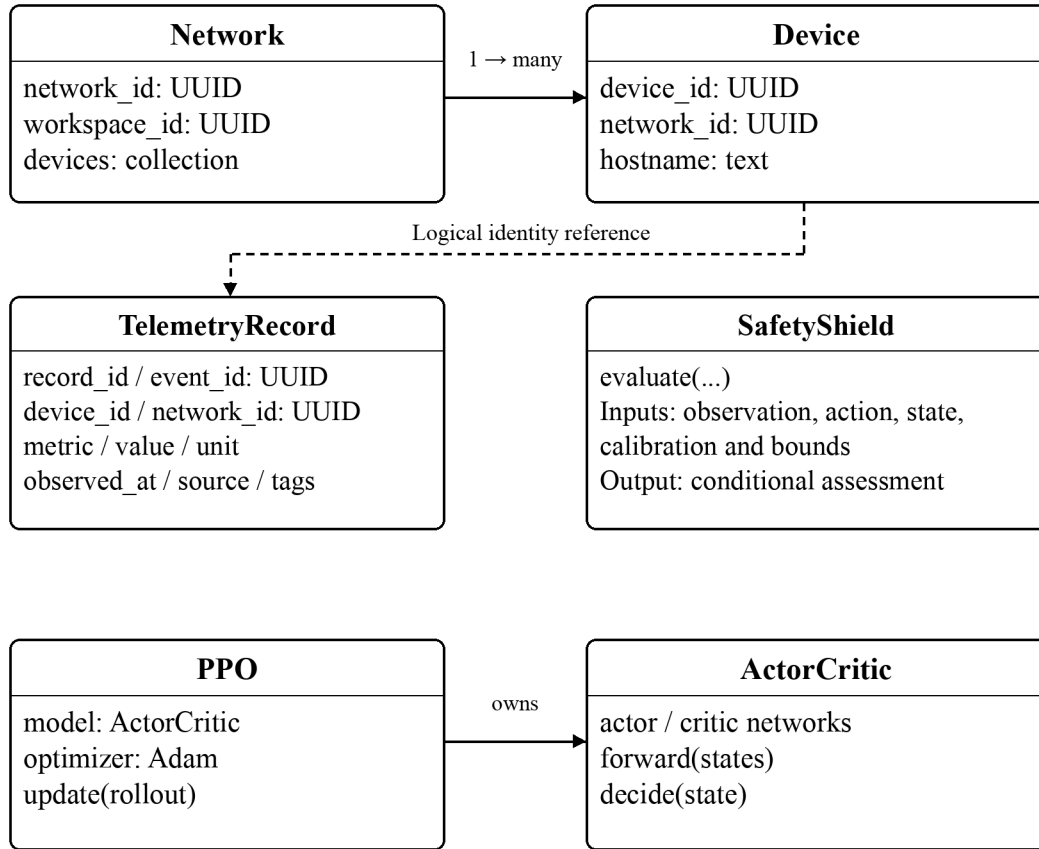


Figure 3.2. NANFO operator use cases

3.4.2 Class Diagram

Figure 3.3 presents representative implemented classes. Network and Device belong to the inventory module; TelemetryRecord stores scoped observations. PPO owns the ActorCritic model and its optimizer, while SafetyShield evaluates typed observations, candidate actions and calibrated bounds separately from the optimizer. The diagram deliberately distinguishes logical identity references from object ownership and does not imply that a controller directly queries another module’s database tables.



Representative classes; the safety assessment does not update the PPO weights.

Figure 3.3. Representative implemented classes and ownership

3.4.3 Entity Relationship Diagram

Figure 3.4 shows the core relational entities for inventory and telemetry. The Network module owns both networks and devices, allowing a database foreign key between them. Telemetry records carry device, network and workspace identifiers as logical references; cross-module access remains through owning services or events. Identity owns user and session data. Passwords are stored as hashes rather than reversible encrypted credentials, and topology graph relationships are maintained in Neo4j rather than represented as cross-module SQL joins.

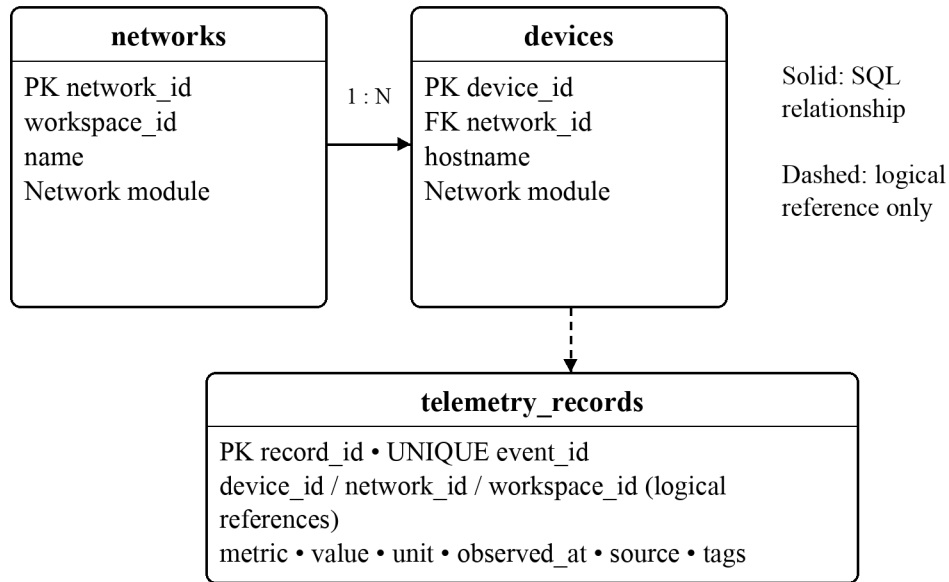


Figure 3.4. Core inventory and telemetry entity relationships

3.4.4 Database Schema

Table 3.1 gives representative columns from the implemented PostgreSQL models. It replaces the earlier illustrative MySQL schema with the current data types and ownership rules. Observations preserve their source, timestamp and unit so that historical reports can distinguish measured values from model outputs. The full schema also includes module-owned intent, simulation, alert, report and audit records.

Table 3.1: Representative relational schema specifications

Table and column	Type and constraint	Purpose
networks.network_id	UUID, primary key	Identifies a managed network.
devices.device_id	UUID, primary key	Identifies an inventory device.
devices.network_id	UUID, within-module foreign key	Links a device to its owning network.
telemetry_records.record_id	UUID, primary key	Identifies a stored metric.

Table and column	Type and constraint	Purpose
telemetry_records.event_id	UUID, unique, non-null	Supports duplicate-event rejection.
telemetry_records.device_id / network_id / workspace_id	UUID, logical references	Retains observation scope without cross-module SQL foreign keys.
telemetry_records.metric / value / unit	TEXT / floating point / nullable TEXT	Stores metric identity, numerical value and unit.
telemetry_records.observ ed_at / source / tags	Time-zone-aware timestamp / TEXT / JSONB	Records observation time and provenance.

3.4.5 System Architecture

Figure 3.5 separates the operator interface, application modules, persistence, learning runtime and emulated data plane. The FastAPI application owns authorization and workflow state. Ryu controls OpenFlow switches in the operator lab, while the matched Linux/FRRouting experiment has a distinct routing interface. The PyTorch process provides versioned model evaluation and inference. Simulation produces predicted outcomes, and authorized execution plus readback produces evidence of an applied configuration. These paths must remain distinguishable in the interface and reports.

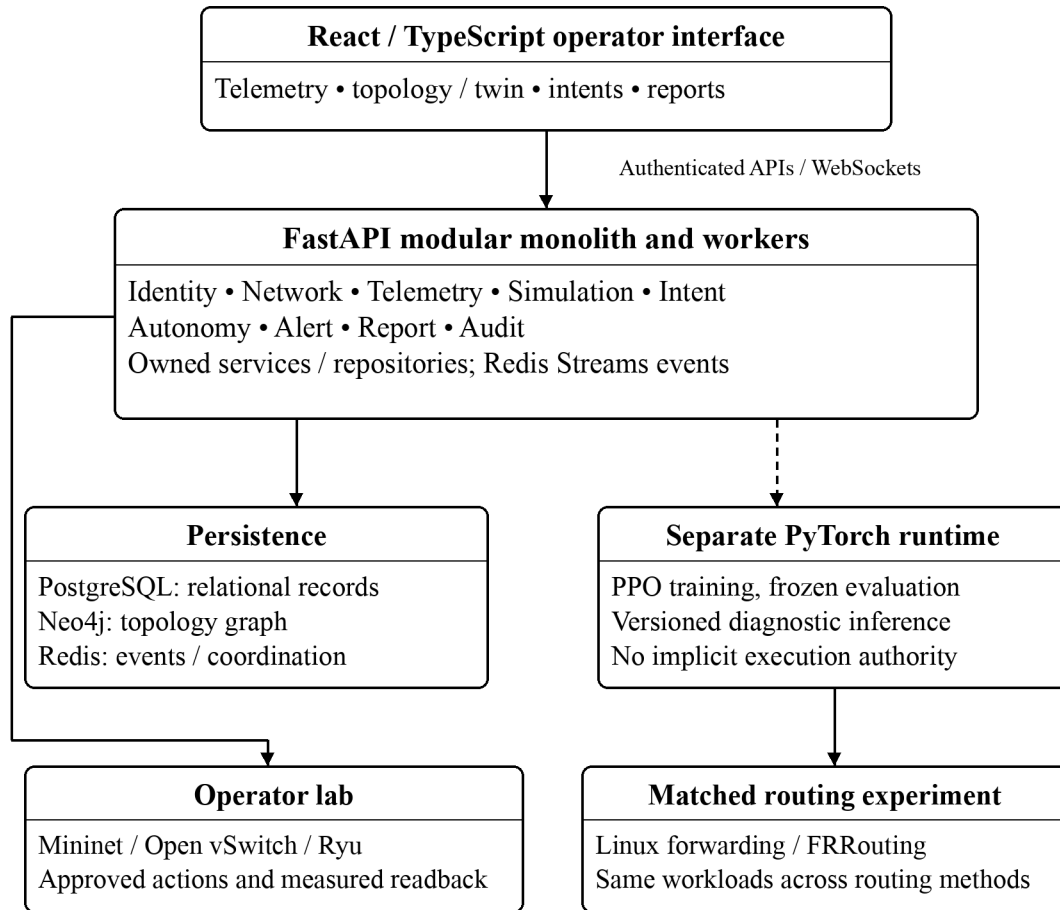


Figure 3.5. Current application and experiment architecture

3.4.6 Wireframes

Figure 3.6 summarizes the operator layout: navigation, active workspace, network condition, topology, metric history and action evidence. The displayed values are schematic placeholders rather than measured results. The interface will retain separate labels for observations, predictions, proposals and verified actions, supporting human review of the closed-loop process.

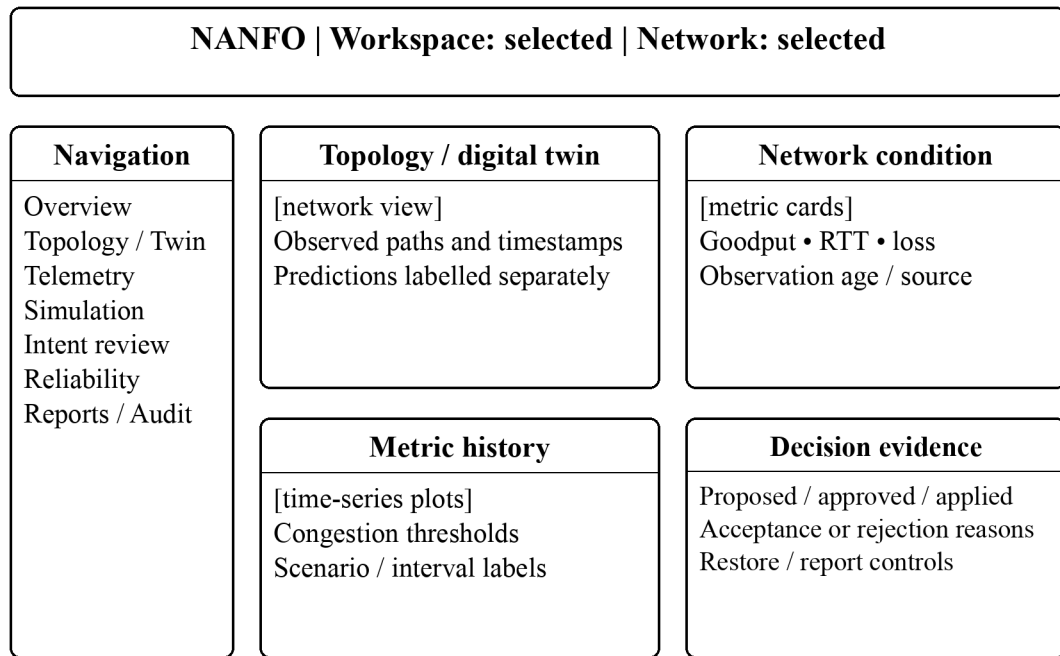


Figure 3.6. Operator dashboard wireframe

3.5 System Development Tools and Techniques

The selected tools support repeatable experimentation and the application’s existing module boundaries. Table 3.2 lists their actual roles. Dependency versions and runtime configuration will be retained with the corresponding release and experiment; compatibility will not be inferred merely because components use the same programming language.

Table 3.2: Software tools and experimental environment

Tool or environment	Role	Selection rationale
Mininet and Open vSwitch	Isolated wired operator testbed	Runs real host and switching code under controlled link settings.
Ryu / OpenFlow 1.3	Operator control plane	Supports explicit flow installation and device statistics.
Linux forwarding and FRRouting	Matched OSPF/learning comparison	Keeps the comparison’s forwarding and background-traffic conditions aligned.

Tool or environment	Role	Selection rationale
PyTorch and Gymnasium	PPO model and environment interface	Supports inspectable tensors, on-policy updates and reproducible checkpoints.
FastAPI	Application backend	Provides typed APIs and module-owned workflows.
React, TypeScript and Three.js	Operator and digital-twin interface	Presents application state, telemetry and spatial views.
PostgreSQL, Neo4j and Redis	Relational, graph and event storage	Separates transactional records, topology queries and event coordination.
Docker and release manifests	Reproducible runtime packaging	Records images, settings and operational dependencies.

3.5.1 Mininet Network Emulator

Mininet will support repeatable network experiments using virtual hosts and links on a single machine. Its use of real network software is suitable for testing controller interaction without a dedicated hardware laboratory (Lantz et al., 2010). Host resources and kernel scheduling remain shared, so the study will document the machine, runtime versions and experiment duration. The topology represents a bounded campus-style network rather than a verified replica of the university.

3.5.2 Ryu Software-Defined Networking Framework

Ryu will provide the OpenFlow control plane for the operator testbed. It will support switch communication and approved flow changes, with Linux instrumentation supplying measurements unavailable from ordinary port counters. The PyTorch model remains outside the Ryu process. FRRouting is used separately for the matched OSPF experiment and is not presented as an OpenFlow application (Bhudia, 2026b).

3.5.3 PyTorch Deep Learning Library

PyTorch will provide tensor operations, automatic differentiation and the actor–critic model. It supports explicit inspection of policy probabilities, value estimates and

optimizer updates. Frozen checkpoints will be loaded with their matching observation contract, and replay checks will compare recorded outputs with model inference. A successful diagnostic replay establishes reproducibility of inference for the supplied observations, not authorization to control the network.

3.5.4 Deep Reinforcement Learning

The implemented technique is categorical PPO, using the two-path action space and the reward described in Section 3.2.3. Training samples actions to explore the isolated environment, while evaluation uses the declared frozen decision rule. A route-change penalty represents one stability preference within the reward. Its effect will be distinguished from the additional execution restrictions through the planned ablation (Schulman et al., 2017).

3.5.5 Conditional Stability Assessment and Operational Safeguards

The current safeguard design assesses a bounded next-step queue envelope rather than applying the earlier continuous-time neural-weight update law. In Equation 3.4, q_i is the observed queue in bytes, a_i^+ is an upper arrival-rate bound, s_i^- is a lower service-rate bound, Δt is the bounded assessment horizon and e_i^+ is an error allowance. All units and bounds must correspond to the same calibrated experiment. This formulation describes the implemented assessment and does not establish its physical assumptions by itself (Bhudia, 2026b).

$$q_i^+ = \max[0, q_i + (a_i^+ - s_i^-)\Delta t] + e_i^+$$

Equation 3.4: Conditional next-step queue envelope

The candidate assessment uses a quadratic queue function V and a configured drift budget B , as expressed in Equation 3.5. Acceptance also requires the current and bounded next queues to remain within configured queue limits. Observation freshness, route validity, calibrated provenance, expiry, minimum holding time and change-rate checks provide additional conditions. A missing or incompatible bound prevents acceptance; a one-step certificate does not independently authorize dispatch or prove indefinite closed-loop stability.

$$V(q) = \frac{1}{2} \sum_i q_i^2; V(q^+) - V(q) \leq B$$

Equation 3.5: Queue function and drift acceptance condition

This separation is consistent with the need for explicit assumptions in safe reinforcement learning (Chow et al., 2018). The application also requires authorized execution, emergency stop, timed override recovery and verified restoration. The numerical simulator supplies model-based evidence, while live calibration and compatibility remain separate acceptance requirements. The study will report which safeguards are implemented, which are measured in the emulator, and which remain unvalidated.

3.5.6 API and Event-Driven Integration

Authenticated REST APIs will expose application workflows to the frontend, and WebSocket updates will support current observations and lifecycle changes. Backend services will access their own repositories and coordinate through documented events or owning-service interfaces. The browser will not directly query the database or act as an independent Ryu control client. This structure supports consistent scope checks, traceable requests and separation of user interaction from long-running experiments.

3.6 System Deliverables

The deliverables combine the software prototype with the evidence required to evaluate it. Completion will be assessed against the four specific objectives, with implemented functionality distinguished from remaining experimental acceptance criteria.

3.6.1 System Documentation

Documentation will include operator and installation guides, module contracts, experimental protocols and limitations. The evaluation package will retain workload plans, model identities, raw measurements and analysis procedures so that reported values remain traceable.

3.6.2 Authentication Module

Authentication will provide password hashing, session management and role-based access. Organization and workspace membership will restrict access to relevant resources. Monitoring, proposing changes and approving execution will remain distinct permissions.

3.6.3 Telemetry Module

Telemetry will retain scoped observations with sources, units and timestamps, support historical queries and update the interface. Measured alerts will record sustained breaches and recovery. Reports will distinguish measured, unavailable and model-derived values.

3.6.4 NANFO Cognitive Routing Module

The routing deliverable will comprise the versioned PPO model, its measurement contract and its connection to the governed decision workflow. It will select candidate paths from observations and record the resulting probabilities and value estimates. Full autonomous operation will only count as delivered when compatible observation, safeguards, authorized actuation and recovery have been validated together.

3.6.4.1 The Actor–Critic Model

The actor–critic component will include saved weights, model configuration and reproducible inference procedures. The retained selected checkpoint provides existing evidence of useful directional routing in the bounded stationary benchmark. It does not imply that the policy has learned the traffic of the university’s live network (Bhudia, 2026a).

3.6.4.2 The Operational Safeguard Layer

The safeguard layer will evaluate the conditions in Section 3.5.5 and retain acceptance or rejection reasons. It will support inspectable policy restrictions, emergency stop and timed recovery workflows. Its effect on route changes and delivery will be measured separately from the learning reward.

3.6.5 Interactive Administrator Dashboard Module

The dashboard will provide topology and digital-twin views, telemetry histories, simulation comparisons, intent review, diagnostics, alerts and audit evidence. It will support approved manual actions, timed overrides and downloadable CSV/PDF reports. The plugin registry manages metadata rather than executing third-party extensions, and the energy function remains an estimator (Bhudia, 2026b).

References

- African Union Commission. (2022). *Digital education strategy and implementation plan 2023–2028*. https://au.int/sites/default/files/documents/42416-doc-1_DES_EN_-_2022_09_14.pdf
- Almasan, P., Suárez-Varela, J., Rusek, K., Barlet-Ros, P., & Cabellos-Aparicio, A. (2022). Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case. *Computer Communications*, 196, 184–194. <https://doi.org/10.1016/j.comcom.2022.09.029>
- Almasan, P., Xiao, S., Cheng, X., Shi, X., Barlet-Ros, P., & Cabellos-Aparicio, A. (2022). ENERO: Efficient real-time WAN routing optimization with deep reinforcement learning. *Computer Networks*, 214, Article 109166. <https://doi.org/10.1016/j.comnet.2022.109166>
- Arista Networks. (2026). *CloudVision data sheet*. https://www.arista.com/assets/data/pdf/Datasheets/EOSCloudVision_DataSheet.pdf
- Bhudia, D. H. (2026a). *Expanded measured PPO outcome* [Unpublished project technical report]. NANFO project, Strathmore University.
- Bhudia, D. H. (2026b). *NANFO development journal* [Unpublished project record; revision consulted September 17, 2026]. NANFO project, Strathmore University.
- Chow, Y., Nachum, O., Duenez-Guzman, E., & Ghavamzadeh, M. (2018). A Lyapunov-based approach to safe reinforcement learning. *Advances in Neural Information Processing Systems*, 31. <https://proceedings.neurips.cc/paper/2018/hash/4fe5149039b52765bde64beb9f674940-Abstract.html>
- Cisco Systems. (2026). *Cisco Catalyst Center 2.3.7 data sheet*. <https://www.cisco.com/c/en/us/products/collateral/cloud-systems-management/dna-center/nb-06-dna-center-data-sheet-cte-en.html>

- Gettys, J., & Nichols, K. (2012). Bufferbloat: Dark buffers in the Internet. *Communications of the ACM*, 55(1), 57–65.
<https://doi.org/10.1145/2063176.2063196>
- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). Deep reinforcement learning that matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 3207–3214.
<https://doi.org/10.1609/aaai.v32i1.11694>
- Juniper Networks. (n.d.). *Marvis conversational assistant*. Retrieved September 17, 2026, from <https://www.juniper.net/documentation/us/en/software/mist/mist-aiops/topics/concept/marvis-conversational-assistant.html>
- Kreutz, D., Ramos, F. M. V., Verissimo, P. E., Rothenberg, C. E., Azodolmolky, S., & Uhlig, S. (2015). Software-defined networking: A comprehensive survey. *Proceedings of the IEEE*, 103(1), 14–76.
<https://doi.org/10.1109/JPROC.2014.2371999>
- Lantz, B., Heller, B., & McKeown, N. (2010). A network in a laptop: Rapid prototyping for software-defined networks. *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, Article 19, 1–6.
<https://doi.org/10.1145/1868447.1868466>
- Li, X., Li, J., Zhou, J., & Liu, J. (2025). Towards robust routing: Enabling long-range perception with the power of graph transformers and deep reinforcement learning in software-defined networks. *Electronics*, 14(3), Article 476.
<https://doi.org/10.3390/electronics14030476>
- Moy, J. (1998). *OSPF version 2* (RFC 2328). Internet Engineering Task Force.
<https://doi.org/10.17487/RFC2328>
- Navarro-Alarcon, D., Qi, J., Zhu, J., & Cherubini, A. (2020). A Lyapunov-stable adaptive method to approximate sensorimotor models for sensor-based control. *Frontiers in Neurorobotics*, 14, Article 59.
<https://doi.org/10.3389/fnbot.2020.00059>

- Rusek, K., Suárez-Varela, J., Mestres, A., Barlet-Ros, P., & Cabellos-Aparicio, A. (2019). Unveiling the potential of graph neural networks for network modeling and optimization in SDN. *Proceedings of the 2019 ACM Symposium on SDN Research*, 140–151. <https://doi.org/10.1145/3314148.3314357>
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal policy optimization algorithms*. arXiv. <https://doi.org/10.48550/arXiv.1707.06347>
- Schwaber, K., & Sutherland, J. (2020). *The Scrum guide: The definitive guide to Scrum: The rules of the game*. <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>

Appendix

Appendix A1: Time Schedule

The original April–December 2026 schedule is retained below as the assessed planning baseline. The revised completion targets are September 2026 for telemetry characterization and the comparative literature review, November 2026 for integrated prototype acceptance, and December 2026 for evaluation and documentation. Earlier completion of individual application components does not by itself close the remaining model, proactivity or safeguarded-autonomy tests.

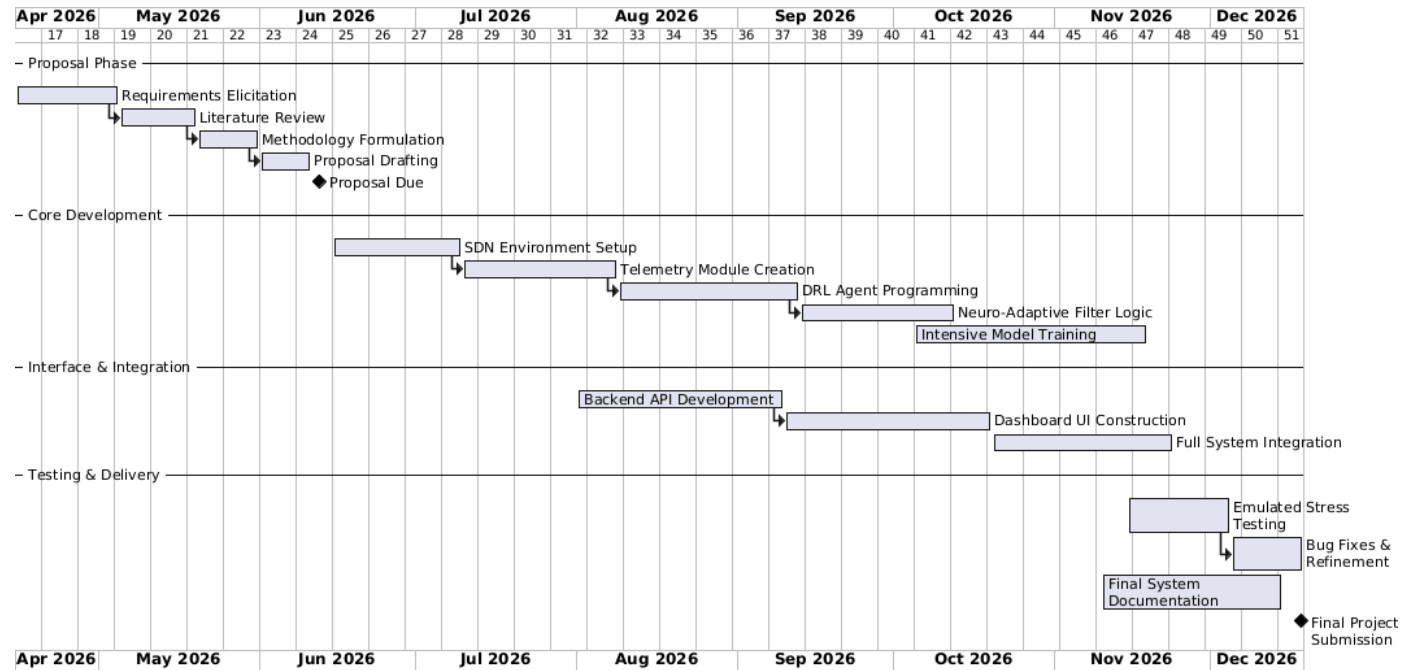


Figure A1. Original project time schedule

Appendix A2: Turn-It-In Report

The following similarity-report images belong to the earlier proposal and are retained as historical records. They do not assess the revised text. A new Turnitin report must replace them after the revised document has been submitted through the university's approved similarity-checking process.

Page 1 of 56 - Cover Page

Submission ID trn:oid::2945:393070704

Dhruvin Bhudia

DhruvinBhudia_169646_MrTiberiusTabulu.pdf

 Strathmore University (Main Account)

Document Details

Submission ID	trn:oid::2945:393070704	49 Pages
Submission Date	Jun 12, 2026, 6:35 PM GMT+3	9,926 Words
Download Date	Jun 12, 2026, 6:40 PM GMT+3	69,699 Characters
File Name	DhruvinBhudia_169646_MrTiberiusTabulu.pdf	
File Size	1.2 MB	

Page 1 of 56 - Cover Page

Submission ID trn:oid::2945:393070704

14% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

• Bibliography

Match Groups

- **85 Not Cited or Quoted 13%**
Matches with neither in-text citation nor quotation marks
- **14 Missing Quotations 1%**
Matches that are still very similar to source material
- **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 7%  Internet sources
- 4%  Publications
- 13%  Submitted works (Student Papers)